



ALGORITHME ET PROGRAMMATION



Par : *Ing. Ulrich DJOKO*

Ingénieur Informaticien et Expert IT

Email : ulrich_djoko@yahoo.com

Tel : 694 965 770

CHAPITRE 1 GENERALITES	3
Introduction.....	3
1. Structures de données linéaires	3
1.1. Les tableaux.....	3
1.2. Les listes chaînées	4
1.3. Les piles et les files.....	5
2. les tables de hachage	5
3. Les arbres.....	6
4. Les graphes	7
4.1 Définition	7
4.2 Structures de données pour la représentation	8
CHAPITRE 2 LISTES CHAINEES.....	9
I- Définition et représentation d'une liste chaînée.....	9
1. Définition.....	9
2. Représentation.....	9
3. Implémentation.....	9
4. Opérations sur les listes	10
II- Liste chaînée simple.....	10
1) Liste Vide	11
2) Ajout d'un élément dans la liste.....	11
3) Création d'une liste.....	13
4) Parcourt d'une liste	14
5) Suppression.....	14
III- Liste Circulaire.....	16
IV- Liste doublement chaînée	16
CHAPITRE 3 PILES ET FILES.....	18
I- Les Piles	18
1. Définition.....	18
2. Représentation.....	18
3. Opérations sur les piles.....	18
a) Initialiser une pile	19
b) Vérifier si une pile est vide.....	19
II- Les Files	20
1. Définition.....	20
2. Représentation.....	20
3. Opérations sur les files.....	20
a) Initialiser une file	21
b) Vérifier si une file est vide	21

PARTIE III : STRUCTURES DE DONNEES AVANCEES

CHAPITRE 1 GENERALITES

Introduction

La façon d'organiser les données traitées par un programme impacte grandement sur la complexité et les performances de ce dernier. La plupart des bons algorithmes utilisent des méthodes astucieuses pour organiser les données. Ces méthodes sont regroupées sous le terme « structures de données ».

Une structure de données est l'implémentation explicite d'un ensemble organisé d'objets, avec la réalisation des opérations d'accès, de construction et de modification afférentes.

On regroupe les structures de données en plusieurs catégories : les structures de données séquentielles, les structures linéaires, les tables, les arbres et les graphes.

En fonction de la méthode de gestion de la mémoire utilisée, on distingue les structures de données statiques des structures de données dynamiques. Ces dernières permettent une gestion plus fine de la mémoire en limitant les gaspillages de mémoire et en facilitant l'évolution de la taille des données.

Le choix de la bonne structure de données dépend surtout du problème et des algorithmes à développer. De plus, on utilise généralement une combinaison de plusieurs types de structures de données pour adresser efficacement un problème.

1. Structures de données linéaires

Les structures de données linéaires sont des structures qui induisent une notion de séquence entre les éléments les composant (1er, 2ème, 3ème, suivant, dernier...).

Parmi les structures de données linéaires on retrouve :

- les tableaux,
- les listes chaînées,
- les piles,
- les files.

1.1. Les tableaux

Les éléments d'un tableau sont de même type et sont placés de façon contigüe en mémoire.

Pour créer un tableau, à 1 ou 2 dimensions, il faut connaître sa taille qui ne pourra être modifiée au cours du programme, et lui associer un indice pour parcourir ses éléments. Pour les tableaux, la séquence correspond aux numéros des cases du tableau. On accède à un élément du tableau directement grâce à son indice.

Soit le tableau à 1 dimension suivant nommé Tablo : 12 14 10 24

Pour atteindre la troisième case du tableau il suffit d'écrire Tablo[3] qui contient 10, si les valeurs de l'indice commencent à 1.

La structure de type tableau pose des problèmes pour insérer ou supprimer un élément car ces actions nécessitent des décalages du contenu des cases du tableau qui prennent du temps dans l'exécution d'un programme.

Ce type de stockage de valeurs peut donc être coûteux en temps d'exécution. Il existe une autre structure, appelée liste chaînée, pour stocker des valeurs. Cette structure permet plus aisément d'insérer et de supprimer des valeurs dans une liste linéaire d'éléments.

Les tableaux sont également dans la catégorie des structures de données séquentielles.

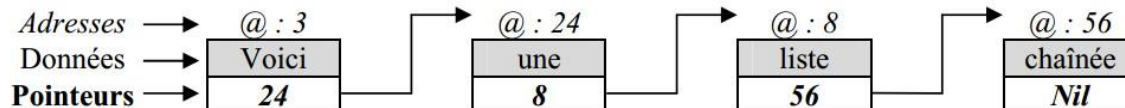
1.2. Les listes chaînées

Une liste chaînée est une structure linéaire qui n'a pas de dimension fixée à sa création. Ses éléments de même type sont éparpillés dans la mémoire et reliés entre eux par des pointeurs. Sa dimension peut être modifiée selon la place disponible en mémoire. La liste est accessible uniquement par sa tête de liste c'est-à-dire son premier élément.

Pour les listes chaînées la séquence est mise en œuvre par le pointeur porté par chaque élément qui indique l'emplacement de l'élément suivant. Le dernier élément de la liste ne pointe sur rien (Nil).

On accède à un élément de la liste en parcourant les éléments grâce à leurs pointeurs.

Soit la liste chaînée suivante (@ indique que le nombre qui le suit représente une adresse):



Pour accéder au troisième élément de la liste il faut toujours débiter la lecture de la liste par son premier élément dans le pointeur duquel est indiqué la position du deuxième élément. Dans le pointeur du deuxième élément de la liste on trouve la position du troisième élément...

Pour ajouter, supprimer ou déplacer un élément il suffit d'allouer une place en mémoire et de mettre à jour les pointeurs des éléments.

Il existe différents types de listes chaînées :

- Liste chaînée simple constituée d'éléments reliés entre eux par des pointeurs.
- Liste doublement chaînée
- Liste circulaire

Ces différents types peuvent être mixés selon les besoins.

On utilise une liste chaînée plutôt qu'un tableau lorsque l'on doit traiter des objets représentés par des suites sur lesquelles on doit effectuer de nombreuses suppressions et de nombreux ajouts. Les manipulations sont alors plus rapides qu'avec des tableaux.

Structure	Dimension	Position d'une information	Accès à une information
Tableau	Fixe	Par son indice	Directement par l'indice
Liste chaînée	Evolue selon les actions	Par son adresse	Séquentiellement par le pointeur de chaque élément

1.3. Les piles et les files

Les files et les piles sont des listes chaînées particulières qui permettent l'ajout et la suppression d'éléments uniquement à une des deux extrémités de la liste.

Structures	Ajout	Suppression	Type de Liste
PILE	Tête	Tête	LIFO (Last In First Out)
FILE	Queue	Tête	FIFO (First In First Out)

La pile est une structure de liste similaire à une pile d'assiettes où l'on pose et l'on prend au sommet de la pile.

La file est une structure de liste similaire à une file d'attente à une caisse, le premier client entré dans la file est le premier sorti de celle-ci (aucun resquillage n'est admis).

2. les tables de hachage

Fonctionnement d'une table de hachage

Une table de hachage est une structure de données qui permet une association clé-élément, où chaque élément est associé à une clé et où chaque clé correspond à un seul élément.

Elle consiste en un tableau (la table de hachage) dont les cases sont appelées alvéoles et une fonction (la fonction de hachage). La fonction de hachage associe à une clé une valeur de hachage. Elle fait donc correspondre la clé d'un élément à une valeur qui est utilisée comme index dans la table de hachage pour cet élément.

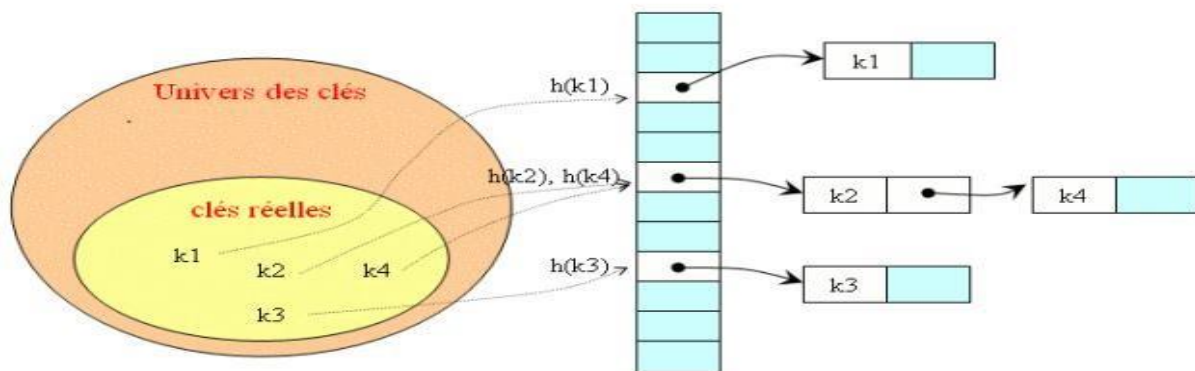
Le but de cette structure de données est d'accéder le plus rapidement possible à un élément à partir de sa clé.

Les opérations permises par une table de hachage sont les suivantes :

- insère(H, élt, clé) : insère dans la table de hachage H un élément élt dont la clef est clé.
- élément recherche(H, clé) : recherche dans la table de hachage H si un élément est associé à la clef clé et renvoie cet élément.
- appartient(H, clé) : recherche dans la table de hachage H si un élément est associé à la clef clé.

Lorsque deux clés ont la même valeur de hachage, on dit qu'on a une collision. Ces clés ne peuvent être stockées à la même position, on doit alors employer une stratégie de résolution

des collisions. Il existe deux méthodes principales pour gérer le problème des collisions : par chaînage ou par adressage ouvert.



3. Les arbres

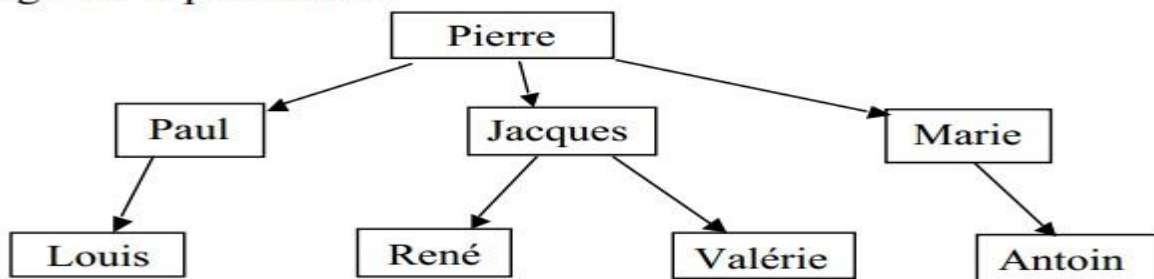
3.1 Définition

L'arbre est une structure de donnée qui généralise la liste : alors qu'une cellule de liste a un seul successeur (leur suivant), dans un arbre il peut y en avoir plusieurs. On parle alors de nœud (au lieu de cellule).

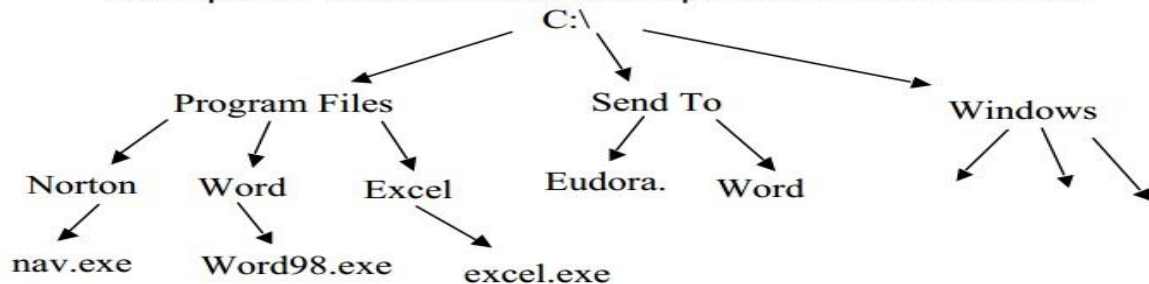
Un nœud père peut avoir plusieurs nœuds fils. **Un fils** n'a qu'un seul père, et tous les nœuds ont un ancêtre commun appelé **la racine** de l'arbre (le seul nœud qui n'a pas de père). **Une feuille** est un nœud qui n'a pas de fils.

3.2 Exemples

Exemple 1 : arbre généalogique
signifie "a pour enfant"



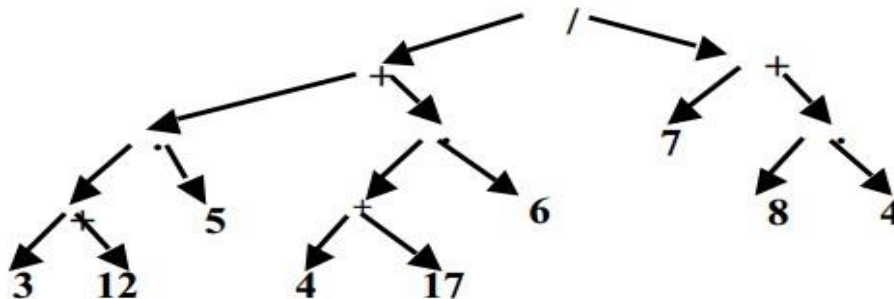
Exemple 2 : arborescence des répertoires et des fichiers



3.3 Exercice

Un arbre de calcul arithmétique est un **arbre binaire** dont tous les nœuds non feuille ont exactement deux fils, et dont les données sur les feuilles sont des nombres et les données sur les nœuds non feuille sont des signes d'opération.

Exemple :



Dessiner l'arbre de calcul arithmétique pour chacune des expressions suivantes :

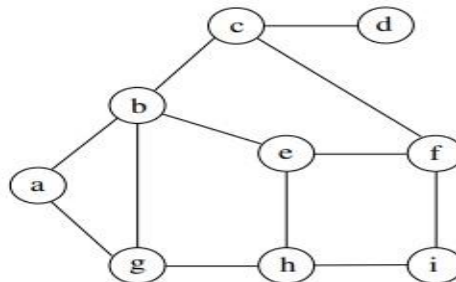
- a) $((3 + 12) * 5 + (4 + 17) * 6) / (7 + 8 * 4)$
- b) $5 \% 4 + 6 * 5 - 89 / 40 + 100 - 2 * 9$

4. Les graphes

4.1 Définition

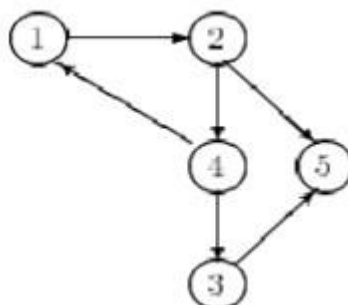
Définition 1 (Graphe non orienté)

Un graphe non-orienté est un couple $G = (X, A)$ où X est l'ensemble des sommets et A est l'ensemble d'arêtes de G . Chaque arête est une paire de sommets. On notera xy l'arête $\{x, y\}$.



Définition 2 (Graphe orienté)

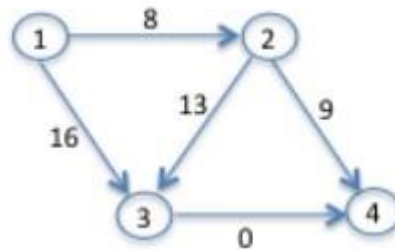
Un graphe orienté est un couple $G = (X, U)$ où X est l'ensemble des sommets et A est l'ensemble d'arcs de G . Chaque arête est un couple de sommets. On notera (xy) l'arête (x, y) .



Remarque 1

Un graphe est valué si des valeurs sont associées à ses arcs.

Exemple de graphe valué :



Remarque 2

Dans un graphe non orienté $G = (X, A)$, l'ordre des extrémités des arêtes ne compte pas, l'arête xy est identique à l'arête yx . Dans un graphe orienté $G = (X, U)$, les arcs (xy) et (yx) n'ont pas la même signification.

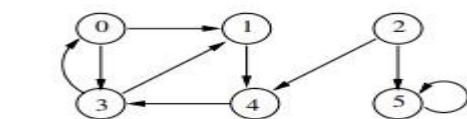
Définition 4 (adjacence, incidence, voisinage, degré)

Deux sommets x et y de $G = (X, A)$ sont adjacents si $xy \in A$. Deux arêtes sont incidentes si elles partagent une même extrémité. L'arête xy est incidente aux sommets x et y . Le voisinage d'un sommet x est $\Gamma(x) = \{y \in X \mid xy \in E\}$. On note $d(x)$ de degré de x , i.e. le nombre de voisins de x .

4.2 Structures de données pour la représentation

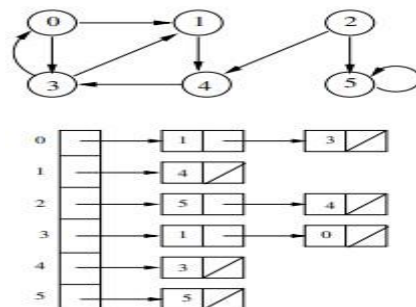
En pratique on représente les graphes par des matrices d'adjacences ou par des tableaux de liste de successeurs.

Exemple avec matrice d'adjacence:



M	0	1	2	3	4	5
0	0	1	0	1	0	0
1	0	0	0	0	1	0
2	0	0	0	0	1	1
3	1	1	0	0	0	0
4	0	0	0	1	0	0
5	0	0	0	0	0	1

Exemple avec liste de successeur :



Domaine d'application : Concernant les très nombreuses applications des graphes, on peut citer : les réseaux électriques et transport d'énergie, le routage du trafic dans les réseaux de télécommunications et les réseaux d'ordinateurs, , les problèmes de localisation (d'entrepôts, d'antennes, . . .) et de placement, les problèmes d'ordonnancement de tâches et d'affectation de ressources, . . .

CHAPITRE 2 LISTES CHAINEES

I- Définition et représentation d'une liste chaînée

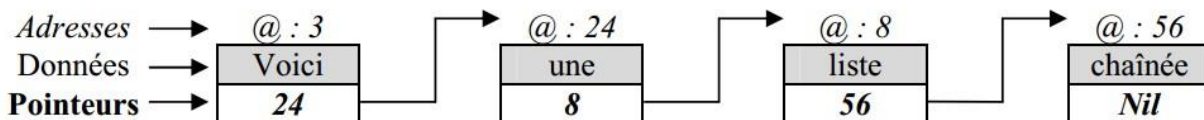
1. Définition

Une liste chaînée est simplement une liste d'objets de même type dans laquelle chaque élément contient :

- des informations relatives au fonctionnement de l'application, par exemple des noms et prénoms de personnes avec adresses et numéros de téléphone pour un carnet d'adresse, - l'adresse de l'élément suivant ou une marque de fin s'il n'y a pas de suivant.

C'est ce lien via l'adresse de l'élément suivant contenue dans l'élément précédent qui fait la "chaîne" et permet de retrouver chaque élément de la liste.

2. Représentation



Typiquement un élément d'une liste chaînée, appelé aussi une cellule ou un maillon, est défini par un enregistrement (structure). Cet enregistrement contient les informations en relation avec les objectifs de l'application et un pointeur sur l'enregistrement suivant de même type : `Element = Enregistrement info : Type de l'information traitée ; suivant : ^Element ;`

`FinEnregistrement`

Remarque : On utilise aussi le terme de structures récursives pour désigner les structures linéaires (listes chaînées, piles, files) et les arbres.

3. Implémentation

On distingue deux implémentations possibles des listes chaînées : l'allocation dynamique de la mémoire et l'allocation contiguë de la mémoire.

a) Allocation dynamique

Les éléments de la liste sont dispersés en mémoire centrale, l'espace est réservé au fur et à mesure de la demande grâce aux primitives de gestion de la mémoire (Allouer et Libérer).

b) Allocation contiguë

Les éléments de la liste sont contiguës soit dans un tableau en mémoire centrale, soit sur fichier en accès direct et dont la taille maximum est spécifiée et contrôlée dans le programme.

Pour une allocation contiguë de mémoire les pointeurs sont des entiers qui correspondent à des indices de tableau ou des positions dans un fichier. La structure de données doit préciser la taille maximum de la liste et définir deux valeurs :

- une valeur de fin de liste
- une valeur qui indique si une position est libre ou pas

4. Opérations sur les listes

Trois opérations principales sont possibles sur les listes :

- Parcours de la liste
- Ajout d'un élément
- Suppression d'un élément □ Savoir si une liste est vide

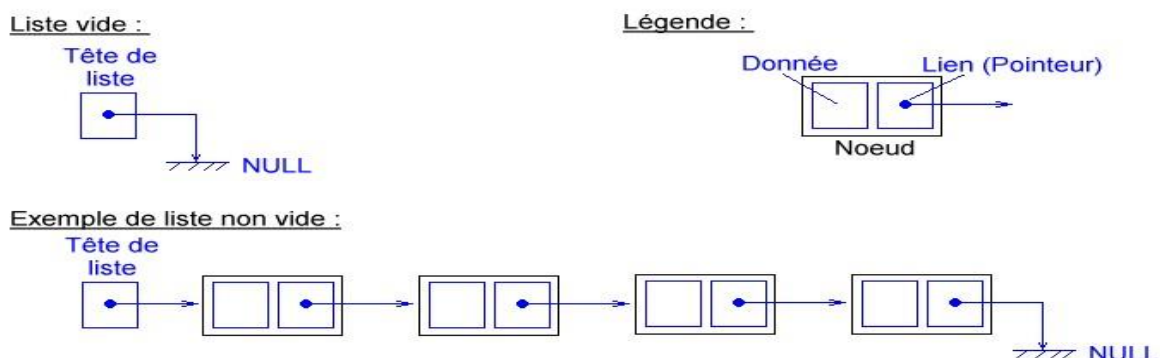
A partir de là d'autres opérations vont être obtenues : recherche d'une donnée, remplacement, concaténation de liste, fusion de listes, etc.

Exercice : Donner les avantages et les limites des listes chaînées par rapport aux tableaux.

II- Liste chaînée simple

Dans une liste simple,

- chaque cellule contient en plus des données, un pointeur vers l'élément suivant de la liste. Le pointeur du dernier élément de la liste a la valeur NIL.
- une variable, appelée **tête ou liste ou premier**, contient l'adresse du premier élément de la liste chaînée.
- lorsque la liste est vide, la tête contient la valeur NIL. □ l'adresse du premier élément définit la liste.



Structures de données utilisées dans cette section

Type Nombre = Enregistrement

info : entier ;

suivant : ^Nombre ;

FinEnregistrement

Variable liste : ^Nombre ; // déclaration de notre liste chaînée de nombres

1) Liste Vide

Principe : Une liste est vide si sa tête contient la valeur NIL.

fonction liste_vide(Entree premier : ^Nombre) : booleen

Debut

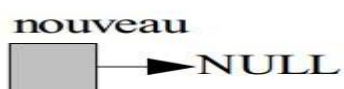
liste_vide $\leftarrow$ (premier = NIL) ;

Fin

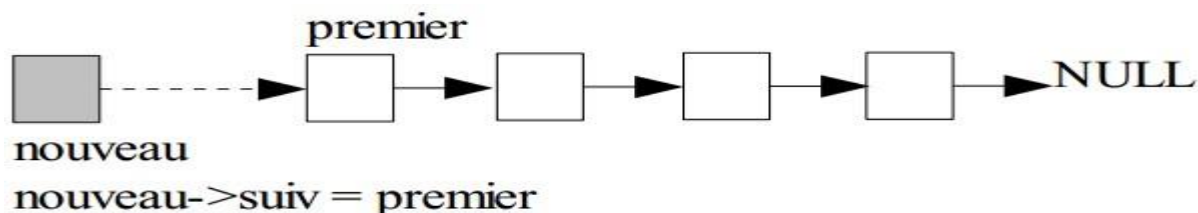
2) Ajout d'un élément dans la liste

a) Ajout d'un nouvel élément en tête de liste

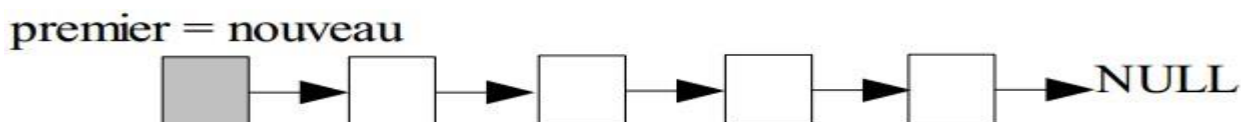
Etape 1 Commencer par créer une nouvelle cellule et initialiser son champ info.



Etape 2 : Ensuite, le nouvel élément est ajouté au début de la liste : son suivant pointe sur le premier élément de la liste.



Etape 3 : Enfin, le pointeur "premier" qui donne l'adresse de la liste, ne contient plus l'adresse du début qui est maintenant dans nouveau, premier doit donc prendre cette adresse :



Procédure ajout_en_tete(Entree/Sortie premier : ^Nombre, val : entier)

Variable nouveau : ^Nombre ;

Debut

Alouer(nouveau) ;

nouveau^.info $\leftarrow$ val ;

nouveau^.suivant $\leftarrow$ premier ;

premier $\leftarrow$ nouveau ;

Fin

b) Ajout d'un nouvel élément en fin de liste

Etape 1 : Commencer par créer une nouvelle cellule et initialiser son champ info.

Etape 2 : Ensuite deux cas se présentent : la liste est vide ou pas.

- Si la liste est vide le premier prend la valeur du nouveau :
- Sinon se positionner sur le dernier élément de la liste et ajouter le « nouveau » à la suite.

procédure ajout_en_fin(Entree/Sortie premier : ^Nombre, val : entier) *variable*

dernier, nouveau : ^Nombre ;

Debut

Alouer(nouveau) ;

nouveau^.info $\leftarrow$ val ;

nouveau^.suivant $\leftarrow$ NIL ;

si(premier = NIL) alors

premier $\leftarrow$ nouveau ;

sinon

dernier $\leftarrow$ premier ; // voyez comme on recherche le dernier élément de la liste

tantque (**dernier^.suivant != NIL**) faire

dernier $\leftarrow$ dernier^.suivant ; // on incrémente dernier

fintanque

dernier^.suivant $\leftarrow$ nouveau ;

finsi

Fin

3) Création d'une liste

Dans ce qui suit, nous allons créer une liste chaînée d'entiers à partir des données fournies par l'utilisateur. Le principe général reste le même, quel que soit la source des données (tableau, fichiers, ...)

On peut créer une liste chaînée par ajout de nouveaux éléments en tête ou en fin de liste.

a) Création d'une liste en faisant des insertions en tête

```
Procédure creation_1(Entree/Sortie premier : ^Nombre)
  Variable nouveau : ^Nombre ;
  Continuer : entier ;
  Debut
    Continuer = 1 ;
    Tantque (continuer = 1) Faire
      Alouer(nouveau) ;
      Ecrire("Entrer un entier : ") ;
      Lire(nouveau^.info) ; // insertion en tête de la liste
      nouveau^.suivant ← premier ;
      premier ← nouveau ;
      Ecrire('voulez vous continuer ? 1 = oui, 0 = non ') ;
      Lire(continuer) ;
    FinTantque
  Fin
```

b) Création d'une liste en faisant des insertions en fin

```
Procédure creation_2(Entree/Sortie premier : ^Nombre)
  Variable nouveau, dernier : ^Nombre ;
  Continuer : entier ;
  Debut
    continuer = 1 ;
    Tantque (continuer = 1) Faire
      Alouer(nouveau) ;
      Ecrire("Entrer un entier : ") ;
      Lire(nouveau^.info) ;
      // insertion en fin de la liste
      si(premier = NIL) alors
        premier ← nouveau ;
      sinon
        dernier ← premier ;
      // voyez comme on recherche le dernier élément de la liste
      tantque (dernier^.suivant != NIL) faire
        dernier ← dernier^.suivant ; // on incrémente dernier
      fintantque
      dernier^.suivant ← nouveau ;
    finsi
    Ecrire('voulez vous continuer ? 1 = oui, 0 = non ') ;
    Lire(continuer) ;
  FinTantque
  Fin
```

4) Parcours d'une liste

Pour parcourir une liste chaînée, il suffit avec un pointeur de prendre successivement l'adresse de chaque élément de la liste.

Au départ le pointeur prend l'adresse du premier élément qui est l'adresse de la liste, ensuite il prend l'adresse du suivant et ainsi de suite.

procedure imprimer_liste(entree premier : ^Nombre)

Variable p : ^Nombre ;

Debut

 P ← premier ;

 Tanque (p != NIL) Faire

 Ecrire (p^.info) ;

 P ← p^.suivant ;

 FinTanque

Fin

5) Suppression

Nous allons distinguer ici la suppression du premier élément de la liste et la suppression du dernier élément de la liste.

a) Suppression en tête de liste

On suppose dans les étapes suivantes que la liste n'est pas vide.

Etape 1 : on initialise un pointeur « p » sur le premier élément de la liste

Etape 2 : on fait pointer le premier élément de la liste sur son suivant

Etape 3 : On libère l'emplacement mémoire de p.

procedure supprimer_en_tete(entree premier : ^Nombre)

Variable p : ^Nombre ;

Debut

 Si (premier != NIL) Alors

 P ← premier ; // etape1

 premier ← premier^.suivant ; // etape2

 Liberer(p) ; // etape 3

 FinSi

Fin

b) Suppression en fin de liste

On suppose dans les étapes suivantes que la liste n'est pas vide.

On distingue deux cas de figure :

Cas ou la liste a un seul élément :

- Etape 1 : on supprime la zone pointée par le premier élément de la liste
- Etape 2 : le premier élément prend la valeur NIL.

Cas ou la liste a plus d'un élément :

- Etape 1 : on trouve l'avant dernier élément de la liste
- Etape 2 : on l'utilise pour supprimer le dernier
- Etape 3 : on met le suivant de l'avant dernier à NIL

procédure
supprimer_en_fin(entree premier : ^Nombre)

Variable avantDer : ^Nombre ;

Debut

si(premier != NIL) Alors

 si(premier^.suivant = NIL) Alors // cas où il ya un seul élément dans la liste

 Liberer(premier) ;

 premier ← NIL ;

 sinon // cas où il y a plus d'un élément dans la liste

 avantDer ← premier ;

 // Voyez comme on recherche l'avant dernier élément de la liste

 Tanque (**avantDer^.suivant^.suivant != NIL**) Faire

 avantDer ← avantDer^.suivant ;

 Fintanque

 Liberer(avantDer^.suivant) ;

 avantDer^.suivant ← NIL ;

fin

FinSi

Fin

6) Exercice

En considérant la liste chaînée d'entiers ci-dessus, écrire les sous programmes suivant :

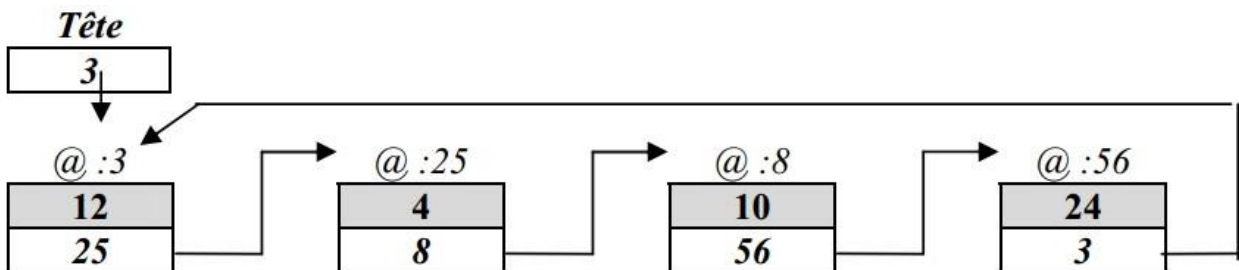
- Fonction qui retourne la taille de la liste (nombre d'éléments de la liste)
- Fonction de recherche booléenne d'un entier donné dans la liste.
- Rechercher et afficher tous les nombre pairs contenus dans cette liste
- Rechercher et afficher le plus petit et le plus grand nombre de la liste
- Procédure qui augmente de « 1 » tous les multiples de 5 présents dans la liste.
- Procédure qui supprime l'entier « 10 » s'il est dans la liste
- Procédure qui supprimer la première occurrence d'un nombre donné de la liste
- Procédure qui ajoute une nouvelle valeur après la première occurrence d'un nombre donné de la liste.
- Procédure qui insère un nouveau nombre à la position « k » de la liste.
- Procédure qui supprime le nombre à la position « k » de la liste.

NB : pour chacune des questions ci-dessus, proposer une version itérative et une version récursive.

III- Liste Circulaire

Une liste chaînée est circulaire lorsque le champ suivant du dernier élément contient l'adresse du premier.

Exemples :



Exemple de liste simplement chaînée circulaire :

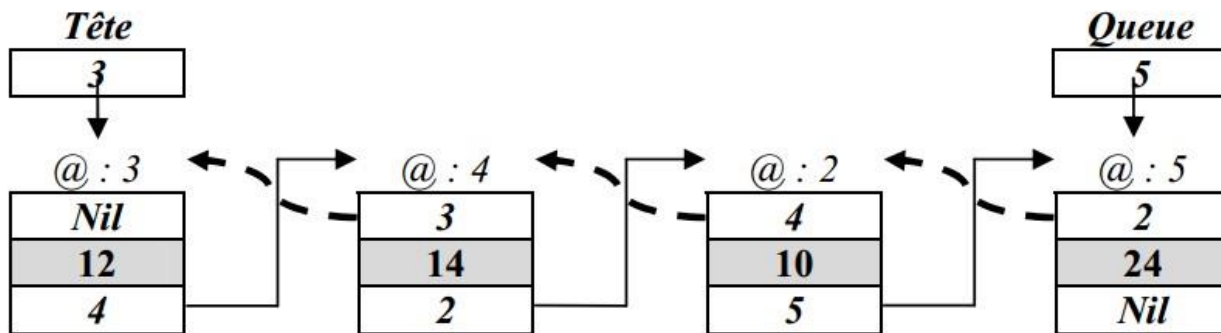
Note : Pour chacun des sous programmes ci-dessous, proposer une version itérative et une version récursive et faire une implémentation en langage C.

IV- Liste doublement chaînée

Une liste doublement chaînée peut-être parcouru dans les deux sens, du premier au dernier et inversement :

- Chaque cellule contient en plus des données, un champ **suivant** qui pointe sur la cellule suivante de la liste, et un champ **precedent** qui pointe vers la cellule précédente de la liste
- Une liste doublement chaînée est repérée par les pointeurs **tête** et **queue**, qui pointent respectivement sur le premier et le dernier élément de la liste.

- Le pointeur « precedent » du premier élément ainsi que le pointeur « suivant » du dernier élément contiennent la valeur NIL.
- Lorsque la liste est vide, « tête » et « queue » ont la valeur NIL.



TD/TP N°1 Gestion d'un carnet d'adresse

Un carnet d'adresses contient des contacts contenant chacun : un nom et un numéro de téléphone.

1. Proposer une structure de liste chaînée pour gérer un carnet d'adresses.
2. Ecrire les sous programmes suivants destinés à gérer cette liste :
 - a) Ajouter un nouveau contact
 - b) Afficher la liste des contacts
 - c) Rechercher tous les numéros de téléphone d'un contact de nom donné
 - d) Mettre à jour un contact donné
 - e) Supprimer un contact du carnet d'adresses

TD/TP N°2 Transport Aérien

Un vol possède un aéroport de départ et une date de départ et un aéroport d'arrivée et une date d'arrivée. Un voyage est constitué de un ou plusieurs vols successifs.

1. Proposer des structures de données pour représenter un voyage.
2. Ecrire une procédure d'enregistrement d'un voyage
3. Ecrire une procédure qui affiche toutes les informations sur un voyage
4. Ecrire une fonction qui donne le nombre d'escales d'un voyage
5. Ecrire une fonction qui retourne la durée (en nombre de jours) d'un voyage
6. Ecrire une fonction qui donne le plus long vol d'un voyage.

TD/TP N°3 Liste de chevaux de course

Ecrire les instructions qui saisissent puis affichent une liste de chevaux de course. Chaque cheval est entré séparément par l'utilisateur. Un cheval est défini par un nom, un dossard, un temps réalisé dans la course, un classement. La liste des chevaux est une liste chaînée. Le programme permet d'ordonner la liste selon le classement des chevaux et de supprimer des chevaux de la liste.

CHAPITRE 3 PILES ET FILES

Les piles et les files sont des formes particulières de structures linéaires dans lesquelles les opérations sont limitées aux extrémités de la structure.

I- Les Piles

1. Définition

Une pile est un ensemble de valeurs ne permettant des insertions ou des suppressions qu'à une seule extrémité, appelée sommet de la pile.

Concrètement, tout nouvel élément est ajouté au sommet de la pile et toute suppression se fait au sommet de la pile. Il s'agit donc d'une structure de type LIFO (Last In First Out).

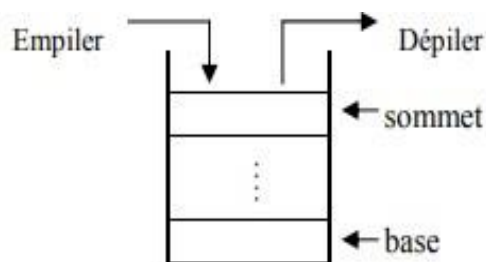
Les piles trouvent de nombreuses applications en informatique notamment en théorie des langages et automates, en compilation, dans les systèmes d'exploitation (pile d'exécution d'un programme, ...), dans l'évaluation d'expressions arithmétiques, ...

2. Représentation

On distingue deux approches pour l'implémentation des piles :

- Les piles statiques : utilisation d'un tableau pour représenter la pile.
- Les piles dynamiques : utilisation des listes chaînées.

Quel que soit l'approche utilisée, on utilise la même notation pour représenter graphiquement une pile :



3. Opérations sur les piles

Les principales opérations possibles sur une pile sont :

- Vérifier si la pile est vide
- Empiler : permet d'ajouter une nouvelle valeur (envoyé en paramètre par l'appelant) à la pile (au-dessus du sommet et dans le cas d'une pile non pleine) ;
- Dépiler : permet de supprimer une valeur (se trouvant au sommet de la pile) et de la renvoyer en paramètre. Cette opération n'est possible que si la pile n'est pas vide

Dans ce qui suit, nous utiliserons une pile d'entier (dans une implémentation dynamique)

Structures de données

Type Element = Enregistrement

Info : entier ;

suivant : ^Element

FinEnregistrement

Type Pile = Enregistrement

sommet : ^Element ;

FinEnregistrement

a) Initialiser une pile

procedure init_pile(entree/sortie pile : Pile)

Debut pile.sommet $\leftarrow$ NIL ;

Fin

b) Vérifier si une pile est vide

fonction pile_vide(entree/sortie pile : Pile) : booleen Debut

pile_vide $\leftarrow$ pile.sommet = NIL ;

Fin

c) Empiler

Empiler un élément revient à faire une insertion en tête dans la liste chaînée.

procedure empiler(entree/sortie pile : Pile, val : entier)

variable p : ^Element ;

Debut

Alouer(p) ; p^.info $\leftarrow$ val ;

p^.suivant $\leftarrow$ pile.sommet ;

pile.sommet $\leftarrow$ p ;

Fin

d) Dépiler

Dépiler revient à faire une suppression en tête en renvoyant la valeur se trouvant au sommet.

Attention ! On ne dépile que si la pile en non vide

fonction depiler(entree/sortie pile : Pile) : entier

variable p : ^Element ; val : entier ;

Debut

// la piste est supposée non vide

p $\leftarrow$ pile.sommet ; val $\leftarrow$ p^.info ;

pile.sommet $\leftarrow$ pile.sommet^.suivant;

Liberer(p) ; depiler $\leftarrow$ val ;

Fin

TPE : Proposer une implémentation statique (avec tableau) d'une pile d'entiers

Par : Ing. Ulrich DJOKO (Ingénieur Informaticien et Expert IT)

II- Les Files

1. Définition

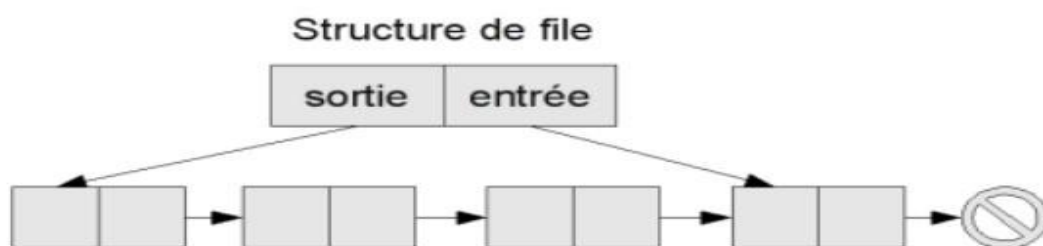
Une file est un ensemble de valeurs qui a un début (Debut) et une fin (Queue).

Les données sont retirées dans l'ordre où elles ont été ajoutées. Concrètement, tout nouvel élément est ajouté à la queue de la file et toute suppression se fait au début de la file. Il s'agit donc d'une structure de type LIFO (Last In First Out).

On retrouve de nombreuses applications des files dans les systèmes d'exploitation (algorithmes d'ordonnancement des processus, liste de travaux à imprimer, ...), dans la construction de systèmes de réservation, dans l'implémentation des applications serveur réseau, les messages en attente dans un commutateur de réseau téléphonique, ...

2. Représentation

On peut implémenter une file dans un tableau (file statique) ou dans une liste chaînée (file dynamique). Quel que soit l'approche employée, on peut utiliser la notation suivante pour représenter graphiquement une file :



3. Opérations sur les files

Les opérations autorisées avec une file sont :

- vérifier si la file est vide ou non.
- enfiler toujours à la queue,
- défiler toujours à la tête si la file n'est pas vide,

Dans ce qui suit, nous utiliserons une file d'entiers (dans une implémentation dynamique)

Structures de données

Type Element = Enregistrement

Info : entier ;

suivant : ^Element ;

FinEnregistrement

Type File = Enregistrement

debut : ^Element ;

queue : ^Element ;

FinEnregistrement

a) Initialiser une file

procedure init_file(entree/sortie file :File)

Debut

file.debut $\leftarrow$ NIL ;

file.queue $\leftarrow$ NIL ;

Fin

b) Vérifier si une file est vide

fonction file_vide(entree/sortie file :File) : boolean

Debut

file_vide $\leftarrow$ file.debut = NIL ;

Fin

c) Enfiler

procedure enfiler (entree/sortie file : File, val : entier)

Variable nouveau : ^Element ;

Debut

Alouer(nouveau)

nouveau^.info = val ;

nouveau^.suivant $\leftarrow$ NIL ;

si (file.debut = NIL) alors // cas où la file est vide

file.debut $\leftarrow$ nouveau ;

file.queue $\leftarrow$ nouveau ;

sinon // cas où la file est non vide

file.queue^.suivant $\leftarrow$ nouveau ;

finsi

Fin

d) Défiler

Attention ! on ne défile que si la file est non vide.

Fonction defiler(entree/sortie file/File) : entier

Variable p : ^Element ;

Val : entier ;

Debut val $\leftarrow$ file.debut^.info ;

p $\leftarrow$ file.debut ;

Si (file.debut = file.queue) alors // cas où il y a un seul élément

file.debut = NIL ;

file.queue $\leftarrow$ NIL ;

sinon // cas où il ya plus d'un élément dans la file

file.debut $\leftarrow$ file.debut^.suivant ;

finsi

Liberer(p) ;

defiler $\leftarrow$ val ;

Fin

TPE : Proposer une implémentation statique (avec tableau) d'une file d'entiers.

Par : Ing. Ulrich DJOKO (Ingénieur Informaticien et Expert IT)